ИССЛЕДОВАНИЕ · NETWORK ATTACK DISCOVERY

Детектирование Mythic C2 с агентом Medusa и профилем HTTP

Сигнатуры Suricata для обнаружения взаимодействия агента Medusa с Mythic C2 через HTTP C2 Profile

АВТОР Белов Т.Ю. (student367, tg: @ares_alive)

ИНСТРУМЕНТЫ Suricata 7.0.14 / 8.0.7, Dalton, Wireshark

ДАТА Октябрь 2026

Оглавление

Получение тестовых сэмплов трафика для создания и проверки правил

Настройка Mythic C2

- Настройка доменов
- Настройка Medusa и HTTP C2 Profile

Захват трафика

Обнаружение IOC

Публичные правила

Отброшенные признаки

Написание правил

- HTTPS
  - JA3/JA4 хеши
  - Сертификат по умолчанию
  - Домен C2 (лабораторные IOC)
  - Периодичность отстука к C2 серверу
  - Энтропия
- HTTP
  - Сообщение Mythic в base64
  - Клиент Medusa
  - Профиль по умолчанию
  - Ответ сервера
  - Периодичность отстука и выгрузка файлов
  - Длина и энтропия параметра
  - UUID импланта (лабораторные IOC)

Проверка правил

Покрытие

False positive / false negative

Q&A

Получение тестовых сэмплов трафика для создания и проверки правил

Для захвата трафика был развернут тестовый стенд, состоящий из двух виртуальных машин Linux и wireshark'a на хосте.

Машина	ОС	IP	Роль
Первая	`Kali`	`192.168.56.100`	Установлен и запущен экземпляр Mythic C2 v3.4.36. DNS-резолвер для второй машины
Вторая	`Parrot`, Python 3.13.5, OpenSSL 3.5	`192.168.56.129`	Таргет для запуска полученной нагрузки из Mythic C2. Ничего лишнего не запущено для сохранения тишины в трафике (кроме включенных по умолчанию системных сервисов). Системный DNS: первая машина. Установлены самоподписанные корневые сертификаты
Хост		`192.168.56.1`	Работают виртуальные машины, стоит wireshark для захвата трафика

Настройка Mythic C2

Настройка доменов

В качестве доменов для отстука агента к C2 были выбраны следующие имена:

- `khenkal.net` - принимает соединения по https на порт 443 с самоподписанным сертификатом
- `ptnad.com` - принимает соединения по http на порт 80
- `mythic.kali.local` - служебный домен для Mythic веб-панели

NGINX, оборачивающий трафик C2, был настроен на получение HTTP/HTTPS трафика на соответствующих доменах

Настройка Medusa и HTTP C2 Profile

Было собрано несколько профилей для получения отклонения в сетевом поведении агента:

- `HTTPS`, GET `q`, POST: `data`, user-agent по умолчанию
- `HTTP`, GET `q`, POST: `data`, user-agent по умолчанию
- `HTTPS`, GET `longquerygetpath`, POST: `veryverylongposturi`, user-agent по умолчанию, **криптография не по умолчанию**
- `HTTPS`, GET `q`, POST: `data`, user-agent по умолчанию, сертификат по умолчанию
- `HTTPS`, GET `q`, POST: `data`, user-agent по умолчанию, сертификат по умолчанию, сервер ограничен TLS 1.2

Везде `encrypted_exchange_check=True`, время между отстуками 3 сек, jitter 3%.

Захват трафика

Трафик был снят с помощью wireshark по схеме 1 дамп - 1 сэмпл.

Дампы `http_local_callback5_3`, `https_local_callback3_1` и `https_ndcrypto_ndpaths_local_callback4_2` содержат несколько передач трафика между таргетом и C2 сервером: download/upload файлов, выполнение команд, завершение работы. Дампы `https_local_mythic_default_crt_callback6_4` и `default_crt_tls12` короткие: checkin и маячки, без передачи файлов.

Из дампов удалён трафик, не относящийся к агенту: широковещательные и мультикаст-пакеты хоста (SSDP, mDNS, UDP 57621 и 21027), ARP, DHCP, ICMPv6, SSH с хоста на C2-сервер и DNS-запросы не к C2-доменам (NTP, `start.parrotsec.org`, служебные запросы Firefox). Фильтр Wireshark:

```
!(ssdp || mdns || udp.port<span class="key">57621 || udp.port</span>21027 || arp || dhcp || icmpv6
|| tcp.port<span class="key">22 || ntp || (dns && !(dns.qry.name </span> "khenkal.net" ||
dns.qry.name == "ptnad.com")))
```

В очищенных дампах остался только обмен между таргетом и C2: DNS к C2-доменам и HTTP/HTTPS. Все цифры ниже и прогоны в Dalton сделаны на очищенных дампах.

Дамп	Профиль	Длительность, с	Пакетов	C2-соединений
`http_local_callback5_3`	HTTP	943	92 552	1342 HTTP
`https_local_callback3_1`	HTTPS	1453	102 276	1505 TLS
`https_ndcrypto_ndpaths_local_callback4_2`	HTTPS, криптография и пути не по умолчанию	312	94 187	1135 TLS
`https_local_mythic_default_crt_callback6_4`	HTTPS, сертификат по умолчанию	70	615	27 TLS
`default_crt_tls12`	HTTPS, сертификат по умолчанию, TLS 1.2	58	407	21 TLS

HTTP-соединения: 308 GET `/index?q=` и 1034 POST `/data`, из них 1024 куска выгрузки файла по ~91 КБ.

Среди полученных данных выделились следующие протоколы:

- DNS (резолвит адрес C2 доменов): на каждое соединение агент заново делает запросы A и AAAA
- HTTP у HTTP профилей
- HTTPS у HTTPS профилей (по TLS 1.3, в `default_crt_tls12` по TLS 1.2)

Других соединений между таргетом и C2 нет: все соединения открывает агент, сервер только отвечает. Каждый запрос агента идёт в новом TCP-соединении (`Connection: close`), по HTTPS в новой TLS-сессии.

Интервал отстука в дампах: 3,06 с, стандартное отклонение 0,04-0,06 с, ~20 запросов в минуту. Во время выгрузки файла до 1049 HTTPS-сессий в минуту.

Шаг обмена ключами (`staging_rsa`) в дампах не виден, несмотря на `encrypted_exchange_check=True`. В HTTP-дампе только два UUID: первый POST (checkin, 436 байт после base64) идёт с UUID `07ce9c45-81ff-4bd6-9d19-82839726d369`, все остальные сообщения идут с UUID `e13c00c5-ca25-48e3-962e-4fffc4488e6a`. При обмене ключами UUID было бы три, а первое сообщение содержало бы RSA-ключ и было бы крупнее. В HTTPS-дампах то же: первая сессия клиента 909 байт (checkin), дальше маячки по 417 байт. Причина не выяснялась.

Обнаружение IOC

- DNS-запросы к доменам (`khenkal.net`, `ptnad.com`)
- Окно отстука повторяется

Для HTTP:

- User-Agent по умолчанию (`Mozilla/5.0 (Windows NT 6.3; Trident/7.0; rv:11.0) like Gecko`)
- Нетипичные заголовки (`Accept-Encoding: identity`, ...)
- PATH по умолчанию (`/index?q=`, `/data`)
- Полезная нагрузка в BODY (`base64` с определенными маркерами)
- Повторяющиеся UUID в параметрах и теле
- Длинный параметр `q` с высокой энтропией
- Заголовки ответа сервера (`Server: NetDNA-cache/2.2`, два `Content-Type`)

Для HTTPS:

- JA3/JA4 хеши агента и C2 сервера
- Домен C2 в SNI
- Сертификат по умолчанию
- Много TLS-сессий в минуту без ALPN

Публичные правила

Перед написанием своих правил были найдены публичные правила по Mythic и проверены на дампах:

Источник	Что проверяет	Результат на дампах
Securelist, sid 9000113	POST: декодирует первые 80 байт тела из base64 и ищет UUID с дефисами	работает: 4 алерта на HTTP-дампе (подавление 1 раз в 3 минуты). Не видит GET (`q=` в base64url) и ответы сервера, на каждый POST вызывает декодирование

Источник	Что проверяет	Результат на дампах
Securelist, sid 9000114	сертификат с `Mythic C2` в TLS-рукопожатии	0. В TLS 1.3 сертификат зашифрован; в TLS 1.2 правило ждёт сертификат в начале TLS-записи, а сервер отправляет его в одной записи с ServerHello
ET Open, sid 2054643	сертификат `O=Mythic` в subject и issuer	0 в TLS 1.3, 21 из 21 сессии в `default_crt_tls12`
ET Open, Blister Loader Mythic C2 Profile M1-M4	пути конкретных кастомных профилей	неприменимы: правила в статусе RETIRED, пути другие
Mjolnir Security, KQL-запрос	регулярность запросов в логах прокси	не правило Suricata. Идея учтена в правилах на частоту

Идея Securelist 9000113 (UUID в начале base64) взята за основу, но проверка сделана без декодирования: по фиксированным символам base64 (см. раздел HTTP). Так правило дешевле и работает на base64url в GET.

Отброшенные признаки

Признак	Почему отброшен
Расшифровка сообщения Mythic	AES-ключ уникален для сборки, сенсору неизвестен
`base64_decode` + поиск UUID в декодированных байтах	дороже проверки символов на фиксированных позициях, не работает с base64url
`Content-Type: application/javascript` в ответе	один из самых частых типов в вебе; используется только косвенно, через дубль `Content-Type`
JA3/JA4 как самостоятельный признак	отпечаток штатного `python3` Debian 13, совпадает у любого Python-скрипта (см. раздел JA3/JA4)
JA3S сервера	зависит от TLS-стека сервера и версии TLS, а не от Mythic
Регулярность интервалов (низкий разброс)	Suricata считает только количество событий за окно; регулярность считается в SIEM/Zeek
Новое TCP-соединение на каждый запрос, `Connection: close`	так работает большинство простых HTTP-клиентов; учтено только в порядке заголовков
Пути `/index`, `/data`, параметр `q` без проверки содержимого	встречаются на тысячах сайтов, оператор меняет их одной настройкой
Энтропия в prod и на Suricata 7	`entropy` есть только в Suricata 8; на реальном трафике есть похожие токены (`cup2key` Google), поэтому только aggressive
UA `Python-urllib`	Medusa подставляет браузерный UA из профиля; `Python-urllib` наоборот исключён из правил, чтобы не ловить обычные скрипты

Написание правил

Правила разбиты на три набора, файлы лежат в `rules/`. Для анализа дампа все они собраны в один файл `AGRESSIVE_FINAL.rules`:

Файл	Правил	Назначение
`prod/prod_http.rules`	7	повседневная работа: открытый HTTP или HTTP после TLS-инспекции
`prod/prod_https.rules`	2	повседневная работа: HTTPS без расшифровки
`aggressive/aggressive_http.rules`	10	дополнение к `prod_http.rules`: максимальное покрытие HTTP
`aggressive/aggressive_http_s8.rules`	1	дополнение к `aggressive_http.rules`, только Suricata 8+ (`entropy`)
`aggressive/aggressive_https.rules`	3	дополнение к `prod_https.rules`: максимальное покрытие HTTPS
`lab_ioc/lab_ioc.rules`	6	IOC стенда (домены, UUID), только для проверки на дампах
`AGRESSIVE_FINAL.rules`	29	все правила выше одним файлом: самое полное детектирование при анализе дампа (включает lab IOC, на боевые сенсоры не ставится)

- `prod` (sid 91xxxxx): в каждом правиле редкая строка для предфильтра и минимум два признака, не больше одного алерта на машину за 5 минут (правила на частоту отстука: раз в минуту);
- `aggressive` (sid 92xxxxx): загружается вместе с prod и только добавляет правила, дублей с prod нет. Без подавления, общие правила на формат сообщения Mythic, низкие пороги частоты, JA4;
- `lab_ioc` (sid 93xxxxx): тестовые домены и UUID сборки стенда, на боевые сенсоры не ставится.

Подключение: prod = `prod_http.rules` + `prod_https.rules`; aggressive = prod + `aggressive_http.rules` + `aggressive_https.rules` (+ `aggressive_http_s8.rules` на Suricata 8). Для HTTPS-наборов в `suricata.yaml` нужно `ja4-fingerprints: yes`. Правила используют `$HOME_NET` и `$EXTERNAL_NET`.

HTTPS

В шифрованном трафике IOC для детектирования Mythic мало.

JA3/JA4 хеши

После вычисления JA3/JA4 хешей был проведен небольшой ресерч (развернут тестовый стенд с Debian 13 и python3). Он показал, что эти хеши полностью совпадают с легитимными отпечатками.

Сами хеши:

- JA3: `284deaeae14ea5ddc96c5d7e59855422`
- JA4: `t13d301200_1d37bd780c83_ecd0401ec68b`

При разборе JA4 хеша видно:

- `t13d301200` - tls 1.3, домен в SNI, 30 наборов шифров, 12 расширений, **00 - нет ALPN**
- `1d37bd780c83` - хеш списка шифров, он определяется OpenSSL
- `ecd0401ec68b` - хеш расширений и подписи

Сравнение с другими клиентами:

Клиент	**JA4**
Агент (все HTTPS-дампы)	`t13d301200_1d37bd780c83_ecd0401ec68b`
`python3` `http.client` в чистом `debian:trixie`	`t13d301200_1d37bd780c83_ecd0401ec68b`
`python3` `urllib.request` в `debian:trixie`	`t13d3014h1_1d37bd780c83_5ae85263eb3b` (есть ALPN `http/1.1`)
`python3` на Manjaro (другая сборка OpenSSL)	`t13d171200_ab0a1bf427ad_ecd0401ec68b`
`curl`	`t13d3013h2_1d37bd780c83_8537cf56674e`

Отпечаток определяется связкой ОС + OpenSSL + библиотека Python, к Mythic он не привязан. Меняется запуском агента на другой ОС или включением ALPN. JA3S сервера (`f4febc55ea12b31ae17cfb7e614afda8` при TLS 1.3, `cca05757f2e4411c7b779f2599c0654d` при TLS 1.2) зависит от TLS-стека сервера и версии TLS, для детекта не годится.

Составленное правило (взят JA4: в нём, в отличие от JA3, видно отсутствие ALPN):

```
alert tls $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa HTTPS beaconing from Debian 13 p
ython3 (JA4 t13d301200_1d37bd780c83_ecd0401ec68b, 20+ per 2 minutes)"; flow:established,to_server; j
a4.hash; content:"t13d301200_1d37bd780c83_ecd0401ec68b"; threshold:type threshold, track by_both, co
unt 20, seconds 120; priority:3; classtype:trojan-activity; sid:9200031; rev:3; metadata:created_at
2026_10_25, updated_at 2026_10_26, ruleset aggressive_https;)
```

Правила на отсутствие ALPN: в разделе «Периодичность отстука к C2 серверу».

Сертификат по умолчанию

Был изучен возвращаемый C2 сервером сертификат по умолчанию:

Subject: `O=Mythic C2`

Issuer: `O=Mythic C2`

Сертификат генерируется профилем при первом запуске (`generateCerts` в `MythicC2Profiles/http`): самоподписанный, ECDSA P-384, без CN и SAN, срок 365 дней, серийный номер случайный. Отпечаток у каждого сервера свой, поэтому детект только по subject/issuer.

Правило для детектирования:

```
alert tls $EXTERNAL_NET any -> $HOME_NET any (msg:"C2 Mythic default self-signed TLS certificate (O=
Mythic C2 in subject and issuer, TLS 1.2 and older)"; flow:established,to_client; tls.cert_subject;
content:"O=Mythic"; fast_pattern; tls.cert_issuer; content:"O=Mythic"; threshold:type limit, track b
y_dst, count 1, seconds 300; classtype:trojan-activity; sid:9100007; rev:4; metadata:created_at 2026
_10_25, updated_at 2026_10_26, ruleset prod_https;)
```

В TLS 1.3 сертификат передаётся зашифрованным, поэтому на трёх дампах с TLS 1.3 правило не сработало. Для проверки снят дамп `default_crt_tls12`: сервер ограничен TLS 1.2, сертификат виден (subject и issuer `O=Mythic C2`, ключ ECDSA). На нём 9100007 совпало на всех 21 сессии и с порогом дало 1 алерт.

Сертификат профиля виден, только если агент подключается к серверу профиля напрямую. Если перед Mythic стоит редиректор (NGINX, CDN) со своим сертификатом, правило его не увидит.

Домен C2 (лабораторные IOC)

Агент на каждое соединение заново резолвит C2-домен (запросы A и AAAA), поэтому для реального C2 правило на домен самое дешёвое и надёжное. **Домены `khenkal.net` и `ptnad.com` тестовые, их нельзя включать в боевые правила.** Правила ниже лежат отдельно в `lab_ioc/lab_ioc.rules` и нужны только для проверки на записанных дампах.

```
alert dns $HOME_NET any -> any any (msg:"C2 Mythic LAB IOC test C2 domain in DNS query (khenkal.net,
do not deploy)"; dns.query; dotprefix; content:".khenkal.net"; nocase; endswith; threshold:type limi
t, track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9300001; rev:1; metadata:creat
ed_at 2026_10_26, updated_at 2026_10_26, ruleset lab_ioc;)
```

```
alert tls $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic LAB IOC test C2 domain in TLS SNI (khen
kal.net, do not deploy)"; flow:established,to_server; tls.sni; dotprefix; content:".khenkal.net"; no
case; endswith; threshold:type limit, track by_src, count 1, seconds 300; classtype:trojan-activity;
sid:9300002; rev:1; metadata:created_at 2026_10_26, updated_at 2026_10_26, ruleset lab_ioc;)
```

```
alert dns $HOME_NET any -> any any (msg:"C2 Mythic LAB IOC test C2 domain in DNS query (ptnad.com, d
o not deploy)"; dns.query; dotprefix; content:".ptnad.com"; nocase; endswith; threshold:type limit,
track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9300003; rev:1; metadata:created_
at 2026_10_26, updated_at 2026_10_26, ruleset lab_ioc;)
```

Периодичность отстука к C2 серверу

Агент стучит на C2 с постоянным интервалом, чтобы получать команды и обновлять статус. На это сделаны правила по частоте соединений.

Браузеры шлют ALPN (`h2`, `http/1.1`), Python `http.client` + `ssl` не шлют. В реальном трафике браузеры открывали до 27 TLS-сессий в минуту к одному хосту, клиенты без ALPN не больше 3, агент около 20.

15+ TLS-сессий в минуту без ALPN (prod):

```
alert tls $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa HTTPS beaconing (15+ TLS session
s per minute to one host, client without ALPN)"; flow:established,to_server; ja4.hash; content:"t";
startswith; content:"00_"; offset:8; depth:3; fast_pattern; threshold:type both, track by_both, coun
t 15, seconds 60; classtype:trojan-activity; sid:9100021; rev:3; metadata:created_at 2026_10_25, upd
ated_at 2026_10_26, ruleset prod_https;)
```

5+ TLS-сессий в минуту без ALPN:

```
alert tls $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa HTTPS beaconing (5+ TLS sessions per minute to one host, client without ALPN)"; flow:established,to_server; ja4.hash; content:"t"; startswith; content:"00_"; offset:8; depth:3; fast_pattern; threshold:type threshold, track by_both, count 5, seconds 60; priority:3; classtype:trojan-activity; sid:9200030; rev:4; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_https;)
```

20+ TLS-сессий за 10 минут без ALPN, медленный отстук:

```
alert tls $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa slow HTTPS beaconing (20+ TLS sessions per 10 minutes to one host, client without ALPN)"; flow:established,to_server; ja4.hash; content:"t"; startswith; content:"00_"; offset:8; depth:3; fast_pattern; threshold:type both, track by_both, count 20, seconds 600; priority:3; classtype:trojan-activity; sid:9200032; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_https;)
```

20+ сессий за 2 минуты с JA4: правило 9200031 выше.

Порог	Ловит интервал отстука до	Правила
15 в минуту	~4 с	9100020, 9100021
20 за 2 минуты	~6 с	9200031
5 в минуту	~12 с (Mythic по умолчанию: 10 с ± 23 %)	9200021, 9200026, 9200030
20 за 10 минут	~30 с	9200025, 9200032

Suricata считает количество событий за окно, а не регулярность. Отстук раз в минуты и реже правилами не ловится, для этого нужен анализ интервалов в SIEM/Zeek (коэффициент вариации интервалов в дампах 0,013-0,019).

Энтропия

В HTTPS не применима: шифртекст TLS у любого клиента имеет максимальную энтропию. Правило по энтропии работает на открытом HTTP, см. раздел HTTP.

HTTP

Пример checkin-запроса и ответа из дампа:

```
POST /data HTTP/1.1
Accept-Encoding: identity
Content-Type: application/x-www-form-urlencoded
Content-Length: 584
Host: ptnad.com:80
User-Agent: Mozilla/5.0 (Windows NT 6.3; Trident/7.0; rv:11.0) like Gecko
Connection: close

MDdjZTljNDUtODFmZi00YmQ2LTlkMTktODI4Mzk3MjZkMzY5OB9r4Fg0ffUPyk7e3oIQpB/j...

HTTP/1.1 200 OK
Cache-Control: max-age=0, no-cache
Content-Length: 412
Content-Type: application/javascript; charset=utf-8
Content-Type: application/octet-stream
Pragma: no-cache
Server: NetDNA-cache/2.2
Connection: close

MDdjZTljNDUtODFmZi00YmQ2LTlkMTktODI4Mzk3MjZkMzY53uUtfngAfFZqx8SN6q4QtzD7...
```

Сообщение Mythic в base64

Любое сообщение агента и сервера имеет вид `base64(UUID + AES-шифртекст)`. UUID4 в виде строки содержит дефисы и цифру версии `4` на фиксированных местах, поэтому в base64 фиксированы символы:

Позиция	0	11	18	24	25	31
Символ	`M` `N` `O` `Y` `Z`	`t`	`0`	`L`	`T` `W`	`t`

Позиции одинаковы для base64 и base64url. Проверено: 400 000 из 400 000 случайных UUID4 совпали, 0 из 200 000 случайных base64-строк. На дампе совпали все 308 GET, 1034 POST и 1342 ответа. **Признак не зависит от путей, UA и ключа шифрования**; проверяется дешёвыми `content` с `offset`/`depth` без декодирования.

Сообщение в начале тела POST, любой агент:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic message from any agent in HTTP POST bo
dy (base64 UUID4 prefix)"; flow:established,to_server; http.method; content:"POST"; http.request_bod
y; content:"t"; offset:11; depth:1; content:"0"; offset:18; depth:1; content:"L"; offset:24; depth:
1; content:"t"; offset:31; depth:1; pcre:"/^[MNOYZ][A-Za-z0-9+\/_-]{10}t[A-Za-z0-9+\/_-]{6}0[A-Za-z0
-9+\/_-]{5}L[TW][A-Za-z0-9+\/_-]{5}t/"; classtype:trojan-activity; sid:9200010; rev:3; metadata:crea
ted_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

Сообщение в параметре:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic message from any agent in HTTP query p
arameter (base64 UUID4 prefix)"; flow:established,to_server; http.uri; content:"="; pcre:"/[?&][^=&]
{1,32}=[MNOYZ][A-Za-z0-9+\/_-]{10}t[A-Za-z0-9+\/_-]{6}0[A-Za-z0-9+\/_-]{5}L[TW][A-Za-z0-9+\/_-]{5}
t/"; classtype:trojan-activity; sid:9200011; rev:3; metadata:created_at 2026_10_25, updated_at 2026_
10_26, ruleset aggressive_http;)
```

Сообщение в ответе сервера:

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"C2 Mythic message from server in HTTP reply body
(base64 UUID4 prefix)"; flow:established,to_client; file.data; content:"t"; offset:11; depth:1; cont
ent:"0"; offset:18; depth:1; content:"L"; offset:24; depth:1; content:"t"; offset:31; depth:1; pcr
e:"/^[MNOYZ][A-Za-z0-9+\/_-]{10}t[A-Za-z0-9+\/_-]{6}0[A-Za-z0-9+\/_-]{5}L[TW][A-Za-z0-9+\/_-]{5}t/";
classtype:trojan-activity; sid:9200015; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_2
6, ruleset aggressive_http;)
```

Дополнительно (в наборы не входят). В профиле `httpx` оператор сам выбирает, куда положить сообщение: в путь, cookie или заголовок. Medusa с профилем `http` так не делает, такого трафика у нас нет, поэтому правила ниже не проверены и в `rules/` не включены. Их можно взять за основу, если в сети встретится Mythic с `httpx`. На Suricata 7 они дорогие: у них нет строки для предфильтра, и на HTTP-дампе каждое проверялось 71 915 раз против 1342 на Suricata 8.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic message from any agent in HTTP URI pat
h (base64 UUID4 prefix)"; flow:established,to_server; http.uri; pcre:"/\/[MNOYZ][A-Za-z0-9_-]{10}t[A
-Za-z0-9_-]{6}0[A-Za-z0-9_-]{5}L[TW][A-Za-z0-9_-]{5}t[A-Za-z0-9_-]{16,}/"; classtype:trojan-activit
y; sid:9200012; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_htt
p;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic message from any agent in HTTP Cookie
(base64 UUID4 prefix)"; flow:established,to_server; http.cookie; pcre:"/(?:^|[\x3b ])[^=\x3b]{1,32}=
[MNOYZ][A-Za-z0-9+\/_-]{10}t[A-Za-z0-9+\/_-]{6}0[A-Za-z0-9+\/_-]{5}L[TW][A-Za-z0-9+\/_-]{5}t/"; clas
stype:trojan-activity; sid:9200013; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ru
leset aggressive_http;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic message from any agent in HTTP request
header (base64 UUID4 prefix)"; flow:established,to_server; http.header; pcre:"/: ?[MNOYZ][A-Za-z0-9+
\/_-]{10}t[A-Za-z0-9+\/_-]{6}0[A-Za-z0-9+\/_-]{5}L[TW][A-Za-z0-9+\/_-]{5}t/"; classtype:trojan-activ
ity; sid:9200014; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_h
ttp;)
```

Клиент Medusa

Python `http.client` всегда ставит заголовки в одном порядке: `Accept-Encoding: identity`, `Host`, `User-Agent`, `Connection: close` (у POST после `Accept-Encoding` идут `Content-Type`, `Content-Length`). Браузеры шлют больше заголовков и `gzip, deflate, br`. От обычного Python-скрипта Medusa отличает браузерный UA.

GET с сообщением Mythic в параметре:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa agent task polling over HTTP GE
T (python header order, Mythic message in query parameter)"; flow:established,to_server; http.metho
d; content:"GET"; http.header_names; content:"|0d 0a|Accept-Encoding|0d 0a|Host|0d 0a|User-Agent|0d
0a|Connection|0d 0a 0d 0a|"; startswith; endswith; fast_pattern; http.accept_enc; content:"identit
y"; bsize:8; http.user_agent; content:!"Python-urllib"; http.uri; content:"="; pcre:"/[?&][^=&]{1,3
2}=[MNOYZ][A-Za-z0-9_-]{10}t[A-Za-z0-9_-]{6}0[A-Za-z0-9_-]{5}L[TW][A-Za-z0-9_-]{5}t/"; threshold:typ
e limit, track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9100001; rev:3; metadat
a:created_at 2026_10_25, updated_at 2026_10_26, ruleset prod_http;)
```

POST с сообщением Mythic в теле:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa agent sending data over HTTP PO
ST (python header order, Mythic message in body)"; flow:established,to_server; http.method; conten
t:"POST"; http.header_names; content:"|0d 0a|Accept-Encoding|0d 0a|Content-Type|0d 0a|Content-Length
|0d 0a|Host|0d 0a|User-Agent|0d 0a|Connection|0d 0a 0d 0a|"; startswith; endswith; fast_pattern; htt
p.accept_enc; content:"identity"; bsize:8; http.request_body; content:"t"; offset:11; depth:1; conte
nt:"0"; offset:18; depth:1; content:"L"; offset:24; depth:1; content:"t"; offset:31; depth:1; pcr
e:"/^[MNOYZ][A-Za-z0-9+\/]{10}t[A-Za-z0-9+\/]{6}0[A-Za-z0-9+\/]{5}L[TW][A-Za-z0-9+\/]{5}t/"; thresho
ld:type limit, track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9100002; rev:3; me
tadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset prod_http;)
```

Любой запрос с браузерным UA и порядком заголовков Python (aggressive):

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic possible Medusa HTTP client (browser U
A with python http.client header order)"; flow:established,to_server; http.header_names; content:"|0
d 0a|Accept-Encoding|0d 0a|"; startswith; content:"|0d 0a|Host|0d 0a|User-Agent|0d 0a|Connection|0d
0a 0d 0a|"; endswith; http.accept_enc; content:"identity"; bsize:8; http.user_agent; content:"Mozill
a/"; startswith; classtype:trojan-activity; sid:9200020; rev:3; metadata:created_at 2026_10_25, upda
ted_at 2026_10_26, ruleset aggressive_http;)
```

Профиль по умолчанию

UA, `/index`, `/data` и `q` по отдельности встречаются на обычных сайтах, поэтому идут только в связке с форматом сообщения. Правила ловят любого агента Mythic с непеределанным профилем, не только Medusa.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic agent with default HTTP profile pollin
g tasks over GET (UA IE11, /index?q= with Mythic message)"; flow:established,to_server; http.method;
content:"GET"; http.user_agent; content:"Mozilla/5.0 (Windows NT 6.3|3b| Trident/7.0|3b| rv:11.0) li
ke Gecko"; bsize:61; fast_pattern; http.uri; content:"/index?q="; startswith; pcre:"/^\/index\?q=[MN
OYZ][A-Za-z0-9_-]{10}t[A-Za-z0-9_-]{6}0[A-Za-z0-9_-]{5}L[TW][A-Za-z0-9_-]{5}t/"; threshold:type limi
t, track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9100003; rev:3; metadata:creat
ed_at 2026_10_25, updated_at 2026_10_26, ruleset prod_http;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic agent with default HTTP profile sendin
g data over POST (UA IE11, /data with Mythic message)"; flow:established,to_server; http.method; con
tent:"POST"; http.user_agent; content:"Mozilla/5.0 (Windows NT 6.3|3b| Trident/7.0|3b| rv:11.0) like
Gecko"; bsize:61; fast_pattern; http.uri; content:"/data"; bsize:5; http.request_body; content:"t";
offset:11; depth:1; content:"0"; offset:18; depth:1; content:"L"; offset:24; depth:1; content:"t"; o
ffset:31; depth:1; pcre:"/^[MNOYZ][A-Za-z0-9+\/]{10}t[A-Za-z0-9+\/]{6}0[A-Za-z0-9+\/]{5}L[TW][A-Za-z
0-9+\/]{5}t/"; threshold:type limit, track by_src, count 1, seconds 300; classtype:trojan-activity;
sid:9100004; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset prod_http;)
```

Ответ сервера

Сервер профиля работает как Go reverse proxy: добавляет заголовки из конфига профиля (`Server: NetDNA-cache/2.2`, `Content-Type: application/javascript`) и оставляет `Content-Type: application/octet-stream` от Mythic. Два `Content-Type` libhtp склеивает в `http.header`, поэтому правило смотрит в `http.header.raw`.

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"C2 Mythic server reply via HTTP profile (duplicate Content-Type headers, Mythic message in body)"; flow:established,to_client; http.stat_code; content:"200"; http.header.raw; content:"Content-Type: application/octet-stream|0d 0a|"; fast_pattern; content:"Content-Type:"; content:"Content-Type:"; distance:0; file.data; content:"t"; offset:11; depth:1; content:"0"; offset:18; depth:1; content:"L"; offset:24; depth:1; content:"t"; offset:31; depth:1; pcre:"/^[MNOYZ][A-Za-z0-9+\/]{10}t[A-Za-z0-9+\/]{6}0[A-Za-z0-9+\/]{5}L[TW][A-Za-z0-9+\/]{5}t/"; threshold:type limit, track by_dst, count 1, seconds 300; classtype:trojan-activity; sid:9100005; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset prod_http;)
```

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"C2 Mythic server reply via HTTP profile (Server NetDNA-cache/2.2, Mythic message in body)"; flow:established,to_client; http.server; content:"NetDNA-cache/2.2"; bsize:16; fast_pattern; file.data; content:"t"; offset:11; depth:1; content:"0"; offset:18; depth:1; content:"L"; offset:24; depth:1; content:"t"; offset:31; depth:1; pcre:"/^[MNOYZ][A-Za-z0-9+\/]{10}t[A-Za-z0-9+\/]{6}0[A-Za-z0-9+\/]{5}L[TW][A-Za-z0-9+\/]{5}t/"; threshold:type limit, track by_dst, count 1, seconds 300; classtype:trojan-activity; sid:9100006; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset prod_http;)
```

В aggressive есть баннер без проверки тела (шумит на CDN NetDNA/StackPath):

```
alert http $EXTERNAL_NET any -> $HOME_NET any (msg:"C2 Mythic possible server reply via HTTP profile (Server NetDNA-cache/2.2 banner only)"; flow:established,to_client; http.server; content:"NetDNA-cache/2.2"; bsize:16; fast_pattern; classtype:trojan-activity; sid:9200006; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

Периодичность отстука и выгрузка файлов

15+ GET в минуту (prod):

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa HTTP beaconing (15+ GET per minute to one host, python header order, browser UA)"; flow:established,to_server; http.method; content:"GET"; http.header_names; content:"|0d 0a|Accept-Encoding|0d 0a|Host|0d 0a|User-Agent|0d 0a|Connection|0d 0a 0d 0a|"; startswith; endswith; fast_pattern; http.accept_enc; content:"identity"; bsize:8; http.user_agent; content:"Mozilla/"; startswith; threshold:type both, track by_both, count 15, seconds 60; classtype:trojan-activity; sid:9100020; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset prod_http;)
```

5+ GET в минуту:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa HTTP beaconing (5+ GET per minute to one host, python header order)"; flow:established,to_server; http.method; content:"GET"; http.header_names; content:"|0d 0a|Accept-Encoding|0d 0a|Host|0d 0a|User-Agent|0d 0a|Connection|0d 0a 0d 0a|"; startswith; endswith; fast_pattern; http.user_agent; content:!"Python-urllib"; threshold:type threshold, track by_both, count 5, seconds 60; classtype:trojan-activity; sid:9200021; rev:4; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

20+ GET за 10 минут, медленный отстук:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa slow HTTP beaconing (20+ GET per 10 minutes to one host, python header order)"; flow:established,to_server; http.method; content:"GET"; http.header_names; content:"|0d 0a|Accept-Encoding|0d 0a|Host|0d 0a|User-Agent|0d 0a|Connection|0d 0a 0d 0a|"; startswith; endswith; fast_pattern; http.user_agent; content:!"Python-urllib"; threshold:type both, track by_both, count 20, seconds 600; classtype:trojan-activity; sid:9200025; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

5+ GET в минуту с сообщением Mythic, любой агент:

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic HTTP beaconing from any agent (5+ GET per minute to one host with Mythic message in query)"; flow:established,to_server; http.method; content:"GET"; http.uri; content:"="; pcre:"/[?&][^=&]{1,32}=[MNOYZ][A-Za-z0-9+\/_-]{10}t[A-Za-z0-9+\/_-]{6}0[A-Za-z0-9+\/_-]{5}L[TW][A-Za-z0-9+\/_-]{5}t/"; threshold:type both, track by_both, count 5, seconds 60; classtype:trojan-activity; sid:9200026; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

Выгрузка файла (5+ POST от 50 000 байт в минуту):

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic Medusa file transfer to C2 over HTTP POST (5+ POST of 50000+ bytes per minute, python header order)"; flow:established,to_server; http.method; content:"POST"; http.header_names; content:"|0d 0a|Accept-Encoding|0d 0a|Content-Type|0d 0a|Content-Length|0d 0a|"; startswith; http.content_len; byte_test:0,>=,50000,0,string,dec; threshold:type threshold, track by_src, count 5, seconds 60; classtype:trojan-activity; sid:9200022; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

Длина и энтропия параметра

Значение `q=` у маячка: 150+ символов base64url с энтропией 5,48-5,79 бит/байт (весь URI 5,59-5,87). Ключевое слово `entropy` есть только в Suricata 8, поэтому для Suricata 7 сделано правило по длине и составу символов. Оба только в aggressive: на реальном трафике похожий токен `cup2key` у Google (143-144 символа) не дотянул до порога 7 символов.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic possible message in HTTP GET query (base64url value 120+ chars, 3+ per 5 minutes)"; flow:established,to_server; http.method; content:"GET"; http.uri; bsize:>150; pcre:"/[?&][A-Za-z0-9_\-]{1,16}=(?=[A-Za-z0-9_\-]*[A-Z])(?=[A-Za-z0-9_\-]*[a-z])(?=[A-Za-z0-9_\-]*[0-9])[A-Za-z0-9_\-]{120,}(?:&|$)/"; threshold:type both, track by_src, count 3, seconds 300; classtype:trojan-activity; sid:9200023; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

Suricata 8 (файл `aggressive_http_s8.rules`):

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic possible message in HTTP GET query (URI entropy above 5.3 bits per byte, 3+ per 5 minutes)"; flow:established,to_server; http.method; content:"GET"; http.uri; bsize:>150; content:"?"; pcre:"/[?&][A-Za-z0-9_\-]{1,16}=[A-Za-z0-9_\-]{120,}(?:&|$)/"; entropy: value > 5.3; threshold:type both, track by_src, count 3, seconds 300; classtype:trojan-activity; sid:9200024; rev:3; metadata:created_at 2026_10_25, updated_at 2026_10_26, ruleset aggressive_http;)
```

UUID импланта (лабораторные IOC)

`e13c00c5-ca25-48e3-962e-4fffc4488e6a` это UUID callback'а, новый при каждом запуске агента. UUID сборки `07ce9c45-81ff-4bd6-9d19-82839726d369` идёт только в checkin и одинаков для всех запусков этой сборки. Оба относятся к сборке стенда и тоже лежат в `lab_ioc/lab_ioc.rules`.

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic LAB IOC callback UUID e13c00c5 in HTTP query (do not deploy)"; flow:established,to_server; http.uri; content:"=ZTEzYzAwYzUtY2EyNS00OGUzLTk2MmUtNGZmZmM0NDg4ZTZh"; fast_pattern; threshold:type limit, track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9300004; rev:1; metadata:created_at 2026_10_26, updated_at 2026_10_26, ruleset lab_ioc;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic LAB IOC callback UUID e13c00c5 in HTTP POST body (do not deploy)"; flow:established,to_server; http.request_body; content:"ZTEzYzAwYzUtY2EyNS00OGUzLTk2MmUtNGZmZmM0NDg4ZTZh"; startswith; fast_pattern; threshold:type limit, track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9300005; rev:1; metadata:created_at 2026_10_26, updated_at 2026_10_26, ruleset lab_ioc;)
```

```
alert http $HOME_NET any -> $EXTERNAL_NET any (msg:"C2 Mythic LAB IOC payload UUID 07ce9c45 in HTTP check-in body (do not deploy)"; flow:established,to_server; http.request_body; content:"MDdjZTljNDUtODFmZi00YmQ2LTlkMTktODI4Mzk3MjZkMzY5"; startswith; fast_pattern; threshold:type limit, track by_src, count 1, seconds 300; classtype:trojan-activity; sid:9300006; rev:1; metadata:created_at 2026_10_26, updated_at 2026_10_26, ruleset lab_ioc;)
```

Проверка правил

Все наборы прогнаны в Dalton на очищенных дампах на Suricata 8.0.7 и 7.0.14. Конфиг: стандартный конфиг Dalton + `EXTERNAL_NET: any` (обе стороны в одной подсети), `ja3-fingerprints: yes`, `ja4-fingerprints: yes`, `profiling.rules.limit: 100`. Aggressive проверялся так, как его подключают: prod + aggressive. Правила загрузились без ошибок во всех прогонах. Экспорты заданий лежат в `dalton_exports/`: zip-экспорт Dalton (правила, конфиг Suricata) и папка `results/` с алертами, профилированием правил и логом Suricata. Дампы из архивов удалены ради размера, они лежат в `pcaps/`.

Набор	Дампы	Задание 8.0.7	Сработало / алертов	Задание 7.0.14	Сработало / алертов
prod_http	HTTP	`930c438743eb7bdf`	7 из 7 / 31	`8b87c854890ffec4`	7 из 7 / 29
prod_https	4 × HTTPS	`1814827a054b60a3`	2 из 2 / 24	`5a0084a26ade364c`	2 из 2 / 23
aggressive_http (с prod_http)	HTTP	`5e206a5d6d25bc22`	18 из 18 / 5 680	`4fcc4b1c264a3a1c`	17 из 17 / 5 674
aggressive_https (с prod_https)	4 × HTTPS	`554a36365dd3b4a7`	5 из 5 / 677	`7bd213a3edcdffda`	5 из 5 / 688
lab_ioc	все 5	`899de064adb7e48e`	6 из 6 / 28	`eafeadee3a5dfaa6`	6 из 6 / 28
AGRESSIVE_FINAL	все 5	`b42bac36a75fc812`	29 из 29 / 6 380	`417df1b94d0dabfe`	28 из 28 / 6 389

Сработали все загруженные правила. На Suricata 7.0.14 нет 9200024: `entropy` есть только в Suricata 8, в AGRESSIVE_FINAL Suricata 7 его не загружает, остальные 28 правил работают.

Пример из `alerts.txt` (задание `930c438743eb7bdf`):

```
10/24/2026-22:22:33.712199 [**] [1:9100002:3] C2 Mythic Medusa agent sending data over HTTP POST (python header order, Mythic message in body) [**] [Classification: A Network Trojan was detected] [Priority: 1] {TCP} 192.168.56.129:36808 -> 192.168.56.100:80
10/24/2026-22:22:52.040621 [**] [1:9100005:3] C2 Mythic server reply via HTTP profile (duplicate Content-Type headers, Mythic message in body) [**] [Classification: A Network Trojan was detected] [Priority: 1] {TCP} 192.168.56.100:80 -> 192.168.56.129:50248
```

На 5 минутах обычного трафика рабочей станции (Suricata 8.0.7) **все наборы дали 0 алертов**. Доказательство слабое: открытого HTTP там 4 GET, Python-клиентов из Debian нет.

Нагрузка (число проверок правил после предфильтра):

Набор	Suricata 8.0.7	Suricata 7.0.14
prod_http	14 292	14 320
prod + aggressive, HTTP	22 230	21 926
prod_https	2 709	2 709
prod + aggressive, HTTPS	10 773	10 773
lab_ioc	12 090	12 090
AGRESSIVE_FINAL	45 093	44 817

Разница prod и aggressive по проверкам около 1,5 раза на обеих версиях Suricata. Основную нагрузку на Suricata 7 давали правила на сообщение в пути, cookie и заголовке (по 71 915 проверок каждое), они вынесены из наборов в дополнительные (см. раздел HTTP).

Покрытие

По HTTP aggressive ловит 100 % событий C2 (308 GET, 1034 POST, 1342 ответа), prod ловит каждый тип событий с подавлением повторов. По HTTPS без расшифровки видны только DNS, SNI, отпечаток клиента, частота сессий и, при TLS 1.2 без редиректора, сертификат Mythic.

Что останется необнаруженным:

Вариант	Что перестаёт работать	Что остаётся
HTTPS без TLS-инспекции, отстук раз в минуты	всё, кроме IOC	IOC домена, анализ интервалов в SIEM
Профиль `httpx` с преобразованием сообщения (XOR, префикс)	правила на base64 UUID	порядок заголовков, частота, длина/энтропия
Правка кода агента (заголовки, `requests`)	правила на клиент Medusa	правила на сообщение Mythic
Смена UA, путей, заголовков сервера	правила на профиль по умолчанию	клиент Medusa, сообщение Mythic
Агент на другой ОС/Python, ALPN	JA4 (9200031), правила без ALPN	сообщение Mythic (HTTP)
Свой сертификат, редиректор перед Mythic или TLS 1.3	9100007	
Новая инфраструктура / сборка	lab IOC (9300001-9300006)	

False positive / false negative

sid	Риск FP	На чём
9100007	очень низкий	чужой сертификат с `O=Mythic` в subject и issuer
9100001-9100006	очень низкий	нужно два признака сразу: Python-клиент с браузерным UA / IE11 / сервер NetDNA и base64 от UUID4 в начале данных
9100020	низкий	Python-скрейпер с браузерным UA, 15+ GET в минуту к одному хосту
9100021	низкий-средний	клиенты без ALPN (Python `ssl`, старая Java, мониторинг) с 15+ сессиями в минуту
9200006	средний	любой сайт за CDN NetDNA/StackPath
9200010, 9200011, 9200015	низкий	приложения, кладущие base64 от UUID в начало тела, параметра или ответа
9200020-9200022, 9200025	средний	Python-скрипты с браузерным UA, регулярные запросы, загрузка файлов
9200023, 9200024	средний-высокий	длинные токены в URL: подписанные ссылки, OAuth `state`, трекинг
9200030-9200032	высокий	любой `python3` из Debian 13 / клиенты без ALPN с регулярными подключениями

Откуда FP на легитимном HTTP: профиль Mythic скопирован с обычного трафика CDN (UA, `Server`, `Content-Type`, пути); клиент Medusa использует стандартную библиотеку Python, как у любого скрипта; регулярные запросы есть у мониторинга и API; длинные случайные токены есть в любом вебе. Поэтому в prod каждый такой признак идёт только в связке с форматом сообщения или с высоким порогом частоты.

False negative на записанном трафике: пропущенных событий C2 нет. Отдельные правила пропустили единичные события (9100005 совпало на 1340 из 1342 ответов, причина не разбиралась), эти события ловят 9200006 и 9200015. Правила на тело зависят от `request-body-limit` / `response-body-limit` в libhtp.

Q&A

Почему были рассмотрены 2 варианта управляющих соединений (по HTTP и HTTPS)?

В инфре заказчика могут встречаться различные варианты, компромиссы и уступки, вследствие чего прокси для терминирования tls/ssl трафика могут отсутствовать, идти в обход средств IPS/IDS

Почему не изменялись те или иные параметры в HTTP профиле агента? (например, UA)

Смена UA, путей, имени параметра и заголовков сервера ломает только правила на значения профиля по умолчанию (9100003-9100006, 9200006). Основные правила смотрят на то, что этими настройками не меняется: формат сообщения Mythic (base64 от UUID в начале данных) и клиент Medusa (порядок заголовков Python, Accept-Encoding: identity). Изменённые пути были в дампе https_ndcrypto_ndpaths, но по HTTPS содержимое не видно, поэтому на HTTP это проверено только по устройству правил, а не на трафике.

Можно ли написать хорошее правило для однозначного детектирования Medusa-HTTP в сети?

У меня не получилось, но поискав в интернете уже готовые правила для детектирования я также не нашёл однозначного хорошего решения из-за возможности хорошей (но не лучшей) кастомизации множества опций агента и схожести трафика с легитимным. Найти и заблокировать можно, но есть большая вероятность отвала части сервисов.